BAB II
TINJAUAN PUSTAKA

Bab ini membahas penelitian dan konsep yang mendukung pengembangan sistem pemeriksaan keamanan kode hasil LLM. Pembahasan dimulai dari keamanan kode yang dihasilkan LLM dan dataset yang digunakan untuk mengujinya. Selanjutnya dibahas pemeriksaan kode dengan SAST, kerja sama beberapa agen, perbaikan kode, serta serangan yang dapat memengaruhi hasil pemeriksaan. Bagian terakhir membandingkan penelitian terdahulu dan menjelaskan posisi penelitian ini.

2.1 Keamanan Kode Hasil LLM

2.1.1 Kebenaran dan keamanan kode

Large language model (LLM) dapat digunakan untuk membuat kode dari instruksi bahasa alami. Meskipun kode tersebut dapat dijalankan dan memberikan hasil yang benar, kode belum tentu aman. Sebagai contoh, program dapat menampilkan data sesuai permintaan pengguna, tetapi tidak memeriksa apakah pengguna berhak mengakses data tersebut. Oleh karena itu, keberhasilan program menjalankan tugas dan keamanan program perlu diperiksa secara terpisah.

Untuk mengelompokkan kelemahan kode, dataset yang dibahas pada bab ini menggunakan *Common Weakness Enumeration* (CWE). Setiap CWE menunjukkan suatu jenis kelemahan, seperti pemeriksaan masukan yang tidak memadai atau penggunaan kata sandi di dalam kode. Pengelompokan ini membantu peneliti menentukan jenis kelemahan yang ingin diuji dan membandingkan temuan pada jenis yang sama [Siddiq dan Santos, 2022; Tony dkk., 2023]. Namun, satu program dapat memiliki lebih dari satu kelemahan. Pemeriksaan terhadap satu CWE belum mencakup seluruh masalah keamanan pada program tersebut.

CyberSecEval menilai keamanan LLM dari dua sisi, yaitu kecenderungan menghasilkan kode rentan dan kesediaan membantu serangan siber. Pada

pengujian keluarga Llama 2, Code Llama, dan GPT, sekitar 30% saran kode pada benchmark tersebut dilaporkan rentan. Alat pendeteksi yang digunakan memperoleh presisi 96% dan *recall* 79% berdasarkan pemeriksaan manual [Bhatt dkk., 2023]. Presisi menunjukkan ketepatan temuan, sedangkan recall menunjukkan seberapa banyak kelemahan yang berhasil ditemukan. Angka kemampuan alat pendeteksi tersebut berbeda dari persentase kode rentan yang dihasilkan model.

Hasil penelitian itu juga menunjukkan bahwa kemampuan membuat kode yang lebih baik tidak selalu diikuti oleh keamanan yang lebih baik. Hal serupa ditemukan pada perbandingan tujuh alat LLM oleh Morkonda dkk. Penelitian tersebut menggunakan 81 prompt Python dari LLMSecEval yang mencakup 12 kategori CWE. Alat yang diuji adalah GPT-4o, Claude 3 Sonnet, Gemini, Perplexity AI, Phind-70b, CodeWhisperer, dan watsonx. Dengan pemeriksaan CodeQL dan GPT-4o untuk kategori tertentu, sedikitnya sekitar 73% potongan kode pada setiap alat dinilai memiliki kerentanan [Morkonda dkk., 2026]. Angka ini berlaku pada tugas yang dipilih dalam penelitian tersebut, sehingga tidak menggambarkan seluruh penggunaan LLM.

2.1.2 Pengaruh prompt dan cara pengujian

Penelitian keamanan kode umumnya menggunakan prompt yang disiapkan untuk percobaan atau mengambil kode dari penggunaan nyata. Prompt yang disiapkan peneliti memudahkan pengujian karena tugas yang diberikan kepada setiap model dapat dibuat sama. Namun, bentuk tugas tersebut dapat berbeda dari permintaan pengguna dalam kegiatan pengembangan perangkat lunak.

Penelitian pada dataset DevGPT menggunakan 2.315 potongan kode C, C++, dan C# dari percakapan pengembang dengan LLM. Pemindaian kode dan pemeriksaan manual menemukan 56 kerentanan pada 48 berkas. Penelitian tersebut juga menguji GPT-4.1, GPT-5, dan Claude Opus 4.1 untuk mendeteksi serta memperbaiki kelemahan [Belozerov dkk., 2026]. Berbeda dari pengujian menggunakan prompt keamanan, data DevGPT mencakup kode yang

diberikan pengguna dan kode yang dibuat model. Karena sumber data dan cara pemeriksaannya berbeda, hasilnya tidak dapat langsung dibandingkan dengan persentase kerentanan pada LLMSecEval.

WildCode Revisited juga membahas keamanan kode dari penggunaan ChatGPT yang nyata. Penelitian tersebut menunjukkan bahwa perhatian pengguna terhadap keamanan masih terbatas [Khanmohammadi dkk., 2026]. Sementara itu, Kharma dkk. menemukan bahwa perubahan cara menyusun prompt dapat mengubah jenis kelemahan yang muncul, tetapi belum memberikan penurunan jumlah atau kepadatan kerentanan yang signifikan secara statistik pada pengujian mereka [Kharma dkk., 2026]. Dengan demikian, permintaan untuk membuat kode aman tetap perlu diikuti pemeriksaan kode. Hasil pada bahasa lain, seperti C/C++ dalam LLM-CSEC, juga perlu disesuaikan sebelum digunakan untuk menjelaskan keamanan kode Python [Shahid dkk., 2025].

Cara menyusun benchmark turut memengaruhi hasil pengujian. Hariharan dkk. menunjukkan bahwa komentar, nama pengenal, dan instruksi pada CyberSecEval dapat memberi petunjuk yang mendorong model menghasilkan pola tidak aman. Ketika petunjuk tersebut diubah, jumlah keluaran yang dinilai aman oleh alat pendeteksi juga berubah [Hariharan dkk., 2024]. Selain itu, dataset publik mungkin telah muncul dalam data pelatihan model. Hal ini perlu diperhatikan agar kemampuan pada dataset lama tidak langsung dianggap mewakili kemampuan pada kode yang benar-benar baru [Chen dkk., 2025].

2.1.3 Dataset penelitian

SecurityEval menyediakan prompt dan contoh kode Python rentan yang dikelompokkan berdasarkan CWE. Pemeriksaan `dataset.jsonl` pada versi repositori `6f4fb70` menghasilkan 121 sampel dengan 69 kategori CWE berdasarkan kolom `ID`. LLMSecEval pada versi `39e97f0` berisi 150 prompt dan 18 kategori CWE. Berdasarkan kolom `Language`, 83 prompt berlabel Python dan mencakup 12 kategori CWE [Siddiq dan Santos, 2022; Tony dkk., 2023]. Jumlah ini menunjukkan isi dataset yang diperiksa, bukan jumlah sampel yang sudah ditetapkan untuk percobaan penelitian ini [VERIFIKASI V01; VERIFIKASI V02].

Kedua dataset menyediakan bentuk tugas yang berbeda, yaitu pelengkapan kode dan pembuatan kode dari instruksi bahasa alami. Keduanya juga memiliki 12 kategori CWE Python yang sama. Penelitian ini direncanakan memilih lima sampai delapan kategori dari cakupan tersebut. Pemilihannya perlu mempertimbangkan jumlah sampel, kemampuan Semgrep dan Bandit, serta kemudahan menjalankan pengujian. Sampel yang sama pada kedua dataset perlu diperiksa agar tidak dihitung dua kali. Perbedaan antara 83 prompt berlabel Python pada repositori dan 81 prompt yang digunakan Morkonda dkk. juga masih perlu dijelaskan [VERIFIKASI V03].

CodeSecEval dapat digunakan sebagai dataset tambahan untuk menilai perbaikan kode. Versi tahun 2024 memuat 180 sampel dan 44 jenis kerentanan. Pada revisi September 2026, jumlahnya menjadi 255 tugas Python dengan 77 CWE serta pengujian fungsi dan keamanan [J. Wang dkk., 2024; J. Wang dkk., 2026]. Jika digunakan, versi dataset dan kelengkapan pengujiannya perlu ditentukan terlebih dahulu [VERIFIKASI V04].

Sintesis dan Keterbatasan. Berbagai penelitian menunjukkan bahwa kode hasil LLM masih dapat memiliki kelemahan meskipun berhasil menjalankan tugas. Namun, hasil pengujian dipengaruhi oleh model, bahasa, prompt, dataset, dan alat pemeriksa. Oleh karena itu, penelitian ini menggunakan batasan Python dan pilihan CWE yang jelas. Jumlah sampel, versi dataset, dan cara menentukan kerentanan perlu dicatat agar hasil pengujian dapat diperiksa kembali.

2.2 Pemeriksaan Kerentanan dengan SAST dan LLM

2.2.1 Pemeriksaan kode secara statis

Static application security testing (SAST) adalah pemeriksaan keamanan kode tanpa menjalankan program. Pemeriksaan dapat dilakukan dengan mencari pola tertentu, memeriksa struktur kode, atau mengikuti aliran data. Sebagai contoh, alat dapat memeriksa apakah masukan pengguna dipakai dalam perintah basis data tanpa penanganan yang sesuai. Pendekatan ini memungkinkan masalah ditemukan sebelum program digunakan.

Meskipun demikian, hasil SAST masih dapat keliru. Alat dapat memberi peringatan pada kode yang sebenarnya tidak rentan, yang disebut *false positive*. Sebaliknya, kelemahan yang tidak ditemukan disebut *false negative*. Kesalahan dapat terjadi ketika alat tidak memahami fungsi pustaka, pemeriksaan masukan buatan pengembang, atau hubungan antarbagian program [Z. Li dkk., 2025; Iranmanesh dkk., 2025]. Oleh karena itu, peringatan dari alat perlu diperiksa bersama konteks kode.

Setiap alat juga memiliki cakupan aturan yang berbeda. Kode yang tidak diberi peringatan oleh Bandit masih mungkin ditandai oleh CodeQL, sebagaimana dibahas dalam Multi-LLMSecCodeEval [Sabir dkk., 2026]. Pada penelitian ini, penggunaan Semgrep dan Bandit perlu disertai catatan versi serta aturan yang digunakan. Hasil tanpa peringatan juga perlu dibedakan dari kegagalan membaca kode atau tidak tersedianya aturan untuk suatu CWE. Ketiga keadaan tersebut memiliki arti yang berbeda bagi keputusan gate.

2.2.2 Penggunaan LLM untuk memeriksa hasil SAST

LLM dapat digunakan untuk membantu memahami konteks yang sulit ditangani aturan statis. Salah satu bentuknya adalah pendekatan *neuro-symbolic*, yaitu penggabungan kemampuan model dalam memahami kode dengan analisis berbasis aturan. IRIS menggunakan LLM untuk membantu menentukan sumber data, bagian kode yang menerima data, dan konteks pemakaiannya dalam CodeQL. Pada versi ICLR 2025, IRIS dengan GPT-4 mendeteksi 55 dari 120 kerentanan dalam CWE-Bench-Java, dibandingkan 27 yang ditemukan CodeQL. Rata-rata *false discovery rate*, yaitu proporsi temuan yang dinilai salah menurut ukuran penelitian tersebut, turun dari 90,03% menjadi 84,82% [Z. Li dkk., 2025].

LLM juga dapat diberi tugas yang lebih terbatas, yaitu memeriksa peringatan yang sudah ditemukan alat. Proses ini disebut triase. ZeroFalse menyediakan jejak aliran data, konteks kode, dan informasi CWE kepada LLM untuk membantu menentukan apakah suatu peringatan perlu dipertahankan. Pengujiannya mencakup OWASP Java Benchmark dan proyek nyata [Iranmanesh dkk., 2025]. SAST-Genius juga menggabungkan Semgrep dengan

LLM dan melaporkan pengurangan false positive. Namun, dataset serta pengaturan percobaannya masih perlu diperiksa sebelum hasilnya dipakai untuk memperkirakan peningkatan pada Python [Agrawal dan Ahi, 2025; VERIFIKASI V06].

QASecClaw menggunakan pembagian tugas yang serupa. Semgrep menghasilkan temuan awal, lalu agen filter berbasis Qwen 3.5 Plus memeriksa temuan tersebut bersama kode. Pada 2.740 kasus Java dalam OWASP Benchmark v1.2, jumlah false positive turun dari 560 menjadi 64, tetapi recall juga sedikit menurun. Sistemnya menyediakan bagian untuk menggabungkan bukti pengujian dinamis, tetapi bagian tersebut tidak dijalankan pada percobaan benchmark statis [Ameen dkk., 2026]. Dengan demikian, keberadaan komponen pengujian dalam rancangan belum berarti seluruh temuan telah dibuktikan melalui eksekusi kode.

Berbeda dari penyaringan peringatan, QRS menggunakan LLM untuk membentuk kueri CodeQL dan memeriksa temuannya pada paket Python utuh [Tsigkourakos dan Patsakis, 2026]. Paket utuh menyediakan konteks yang lebih lengkap daripada sebuah fungsi atau potongan kode. Perbedaan ini perlu diperhatikan ketika cara kerja QRS diterapkan pada kode hasil LLM. Sebagian besar penelitian tersebut memakai kode proyek atau benchmark yang telah tersedia, sehingga hasilnya masih perlu diuji pada kode baru yang dibuat model.

2.2.3 Peran SAST dalam pengambilan keputusan

Dalam rancangan penelitian ini, agen Analyse memeriksa kode dengan bantuan Semgrep dan Bandit. Agen Triage kemudian menilai apakah temuan tersebut sesuai dengan kondisi kode. Pemisahan tugas ini diperlukan karena menemukan pola yang mencurigakan dan memutuskan kebenaran suatu temuan merupakan dua pekerjaan berbeda. Hasil pemeriksaan perlu memuat lokasi, jenis CWE, tingkat keparahan, aturan asal, dan alasan yang mendukung temuan.

Jika beberapa alat melaporkan masalah yang sama, temuan perlu digabungkan agar satu kelemahan tidak dihitung berulang. Setiap hasil juga perlu dihubungkan dengan versi kode yang diperiksa. Setelah kode diperbaiki, hasil pemeriksaan versi sebelumnya tidak dapat langsung digunakan untuk menerima

versi baru. Penjelasan LLM dapat membantu memahami masalah, tetapi tetap perlu dibedakan dari bukti hasil pengujian.

Menurut rancangan awal, Triage tidak boleh sendirian membatalkan temuan SAST berkeparahan tinggi. Aturan ini digunakan untuk mencegah model mengabaikan masalah penting tanpa bukti yang cukup. Namun, aturan tersebut juga dapat menyebabkan kode yang sebenarnya benar lebih sering diminta untuk diperbaiki atau ditinjau manusia. Oleh karena itu, pengaruhnya terhadap ketepatan temuan dan beban pemeriksaan perlu diukur. Penentuan tingkat keparahan serta penanganan temuan yang belum pasti masih perlu ditetapkan [VERIFIKASI V07].

Sintesis dan Keterbatasan. SAST membantu menemukan masalah melalui aturan yang dapat diperiksa, sedangkan LLM membantu menjelaskan konteks kode dan menyaring temuan yang keliru. Penggabungan keduanya berpotensi memperbaiki pemeriksaan, tetapi dapat pula menghilangkan temuan yang sebenarnya benar. Penelitian ini karena itu perlu menilai presisi dan recall secara bersamaan. Hasil pada proyek Java atau paket Python utuh juga belum tentu berlaku sama pada potongan kode Python hasil LLM.

2.3 Sistem Multi-Agen untuk Keamanan Kode

2.3.1 Pembagian tugas antaragen

Sistem multi-agen membagi pekerjaan kepada beberapa agen yang saling bertukar informasi. Dalam pemeriksaan keamanan kode, agen dapat bertugas membuat kode, mencari kelemahan, menguji temuan, atau memperbaiki kode. Pembagian ini membantu setiap agen berfokus pada tugas tertentu. Namun, manfaatnya tetap bergantung pada cara agen bekerja sama dan bagaimana hasilnya diperiksa.

Tujuan sistem perlu dibedakan sebelum hasil penelitian dibandingkan. CodeAgent digunakan untuk membantu proses peninjauan kode, sedangkan VulTrial dan MAVUL lebih berfokus pada pemeriksaan kerentanan pada kode yang sudah tersedia [Tang dkk., 2024; Widyasari dkk., 2025; Y. Li dkk., 2025]. Sistem yang berhasil mengenali kelemahan belum tentu sudah mengatur

pembuatan kode, perbaikannya, dan keputusan apakah kode dapat diterima. Perbedaan tugas ini menjadi dasar dalam memilih pembanding untuk penelitian ini.

Beberapa agen dapat menggunakan model yang sama dengan instruksi berbeda. Pilihan lainnya adalah menggunakan model yang berbeda untuk setiap tugas. Instruksi yang berbeda dapat mengarahkan perhatian agen pada bagian kode yang berbeda, tetapi tidak menjamin pendapatnya saling bebas. Jika agen memiliki kelemahan pemahaman yang sama, beberapa agen dapat menghasilkan kesalahan yang sama pula. Karena itu, manfaat penambahan agen perlu dilihat dari perubahan hasil pemeriksaan, bukan dari banyaknya pendapat yang dihasilkan.

2.3.2 Pembuatan, pemeriksaan, dan pengujian kode

AutoSafeCoder menggabungkan tiga agen, yaitu Coding Agent, Static Analyzer Agent, dan Fuzzing Agent. Agen pembuat kode menerima masukan dari pemeriksaan keamanan, kemudian memperbaiki hasilnya. Pada metode versi pertama, Static Analyzer Agent adalah LLM yang diarahkan untuk mencari kelemahan berdasarkan CWE. Bandit digunakan untuk menilai hasil percobaan, sehingga perannya berbeda dari reviewer LLM dalam sistem tersebut.

Dengan GPT-4o, jumlah kode rentan pada SecurityEval turun dari 59/121 menjadi 44/121. Selisihnya sekitar 12,4 poin persentase berdasarkan jumlah sampel. Angka penurunan 13% dalam paper tidak digunakan sebagai penurunan relatif. Sementara itu, skor pass@1 pada HumanEval turun dari 0,9085 menjadi 0,8780 [Nunez dkk., 2024]. Hasil ini menunjukkan adanya perbaikan keamanan yang disertai sedikit penurunan keberhasilan fungsi. Pengujian fuzzing memeriksa kegagalan saat kode dijalankan, tetapi tidak semua kelemahan keamanan menimbulkan kegagalan yang sama.

Multi-LLMSecCodeEval membandingkan sepuluh alur yang mencakup pembuatan kode, deteksi kerentanan, dan perbaikan. Pada SecLLMEval serta SecLLMHolmes, penggabungan beberapa model, CodeQL, dan pemeriksaan lintas-model dilaporkan memberikan hasil yang lebih konsisten daripada kerja

sama LLM tanpa alat statis [Sabir dkk., 2026]. Penelitian tersebut sudah membahas alur dari pembuatan hingga perbaikan kode. Oleh karena itu, alur tersebut tidak dijadikan kebaruan tersendiri dalam penelitian ini. Perbedaan angka pada beberapa bagian paper masih perlu diperiksa sebelum dipakai dalam perbandingan rinci [VERIFIKASI V05].

Hasil tersebut juga menunjukkan perlunya memisahkan manfaat alat dari manfaat kerja sama agen. Jika pemindai dan beberapa reviewer ditambahkan sekaligus, peningkatan hasil belum tentu berasal dari diskusi antaragen. Penelitian ini karena itu merencanakan pembanding berupa LLM tunggal, SAST saja, serta SAST dengan LLM. Tugas, jumlah kesempatan perbaikan, dan anggaran penggunaan model perlu dibuat sebanding atau dijelaskan perbedaannya.

2.3.3 Penggabungan hasil pemeriksaan

Salah satu cara menggabungkan hasil adalah voting mayoritas, yaitu memilih keputusan yang paling banyak diberikan agen. Cara ini mudah diterapkan, tetapi dapat menghasilkan keputusan salah jika sebagian besar agen memiliki kesalahan yang sama. Cara lain adalah debat, yaitu meminta agen membahas perbedaan pendapat sebelum mengambil kesimpulan.

ReasonVul menggunakan tiga cara penalaran, yaitu deduktif, induktif, dan abduktif. Secara sederhana, agen memeriksa kode berdasarkan aturan umum, pola dari contoh, dan dugaan penyebab masalah. Ketiga tugas tersebut menggunakan Phi4, Llama4, dan DeepSeek. Ketika hasilnya berbeda, agen melakukan debat. Pada analisis konflik dalam percobaan PrimeVul, 389 dari 542 konflik diselesaikan dengan benar. Pada JITVUL, sistem memperoleh PairAcc 28,67% [Peng dkk., 2026]. PairAcc menghitung keberhasilan mengenali kode rentan dan versi perbaikannya secara benar sebagai satu pasangan. Ukuran ini berbeda dari akurasi untuk satu potongan kode.

VulTrial menggunakan pembagian peran seperti ruang sidang. Peneliti keamanan mengajukan dugaan kelemahan, penulis kode memberikan penjelasan, lalu moderator dan review board menilai pembahasannya [Widyasari dkk., 2025]. MAVUL juga menggunakan diskusi antarperan untuk memeriksa konteks dan

hubungan antarbagian kode [Y. Li dkk., 2025]. Kedua pendekatan ini memberi contoh cara mempertanyakan temuan sebelum menerimanya. Namun, tugas hakim atau agen penilai masih perlu dibandingkan dengan aturan penerimaan kode pada gate.

Debat tidak selalu menghasilkan keputusan yang benar. Refute-or-Promote menggunakan pemeriksaan bertahap, kritik dari model lain, dan pengujian nyata untuk menolak temuan yang keliru. Laporan kasusnya menunjukkan bahwa kesepakatan beberapa reviewer masih dapat dibantah oleh hasil pengujian [Agarwal, 2026]. Karena itu, persetujuan banyak agen tidak menggantikan kebutuhan akan bukti. Sumber lain tentang kegagalan debat yang baru diketahui dari judulnya belum dipakai sebagai dasar kesimpulan [VERIFIKASI V08].

2.3.4 Arbiter dan biaya penggunaan model

Arbiter adalah bagian yang menentukan keputusan akhir berdasarkan hasil pemeriksaan. Dalam rancangan penelitian ini, arbiter menggunakan aturan non-LLM yang disimpan dalam YAML. Agen memberikan temuan dan penjelasan, sedangkan arbiter menentukan *pass*, *revise*, atau *block*. Keputusan pass berarti kode diterima berdasarkan pemeriksaan yang ditetapkan. Revise berarti kode perlu diperbaiki, sedangkan block berarti kode perlu ditinjau manusia. Temuan berkeparahan tinggi yang belum selesai ditangani mencegah kode memperoleh pass.

Kode yang diterima diberi label *Verified*. Label ini berarti kode telah melewati pemeriksaan gate, bukan jaminan bahwa kode bebas dari seluruh kelemahan. Alasan keputusan perlu dicatat, termasuk aturan yang digunakan, versi kode, dan hasil pengujiannya. Jika respons agen tidak lengkap atau pengujian gagal dijalankan, arbiter perlu memiliki aturan penanganan yang jelas [VERIFIKASI V07; VERIFIKASI V09].

Penggunaan beberapa model juga menambah biaya. Wang menggabungkan model layanan dengan pemeriksa lokal untuk menyeimbangkan hasil deteksi dan biaya [Wang, 2026]. Pada gate, biaya tidak hanya berasal dari pembuatan kode. Pemeriksaan, diskusi, pembuatan pengujian, dan perbaikan

berulang turut memakai token serta waktu. Menjalankan beberapa agen bersamaan dapat memperpendek waktu tunggu, tetapi tidak selalu mengurangi jumlah penggunaan model.

Sintesis dan Keterbatasan. Penelitian terdahulu telah membahas pembagian peran, diskusi antaragen, dan alur perbaikan kode. Namun, banyaknya agen dan kesepakatan pendapat belum menjamin hasil yang benar. Penelitian ini menempatkan aturan arbiter sebagai pengambil keputusan dan menggunakan bukti pemeriksaan sebagai dasar penilaiannya. Manfaat pembagian tugas, pilihan model, serta aturan tersebut perlu diuji bersama biaya dan waktu pemrosesan. Rincian sistem pembanding, termasuk SecAwareCoder, masih perlu diperiksa sebelum pembeda penelitian ditetapkan [VERIFIKASI V04; VERIFIKASI V10].

2.4 Perbaikan Kode Otomatis

2.4.1 Perbaikan berdasarkan hasil pemeriksaan

Perbaikan kode otomatis bertujuan menghilangkan kelemahan tanpa mengubah fungsi yang dibutuhkan pengguna. Sebuah perubahan belum dapat dianggap berhasil hanya karena peringatan alat hilang. Model dapat saja menghapus bagian kode yang bermasalah, tetapi sekaligus menghilangkan fungsi program. Oleh karena itu, kode yang telah diperbaiki perlu diperiksa kembali dari sisi keamanan dan kebenaran hasilnya.

Salah satu cara memperbaiki kode adalah memberikan hasil analisis statis kepada LLM. Feedback-Driven Security Patching menggunakan keluaran Bandit untuk mengarahkan perbaikan kode Python [Alrashedy dkk., 2024]. Static Analysis as a Feedback Loop juga menggunakan Bandit dan Pylint untuk memperbaiki beberapa aspek kualitas kode [Blyth dkk., 2025]. Pada kedua pendekatan tersebut, model memperoleh informasi tentang masalah yang ditemukan alat. Informasi ini membantu model melakukan perbaikan yang lebih terarah daripada hanya diminta memeriksa jawabannya sendiri.

Penggunaan beberapa sumber masukan dibahas dalam penelitian Sriram dkk. Model menerima hasil kompilasi GCC, pemeriksaan CodeQL, dan pengujian dengan KLEE. Pada 1.720 program C/C++ hasil CodeLlama-7B, tingkat cacat

keamanan menurut CodeQL turun dari 58,55% menjadi 22,19%. Percobaan lainnya menggunakan 1.522 program hasil DeepSeek-Coder-1.3B [Sriram dkk., 2026]. Hasil ini mendukung penggunaan alat untuk membantu perbaikan, tetapi alat pengujian C/C++ tersebut tidak dapat langsung digunakan pada potongan kode Python.

LLMLOOP menghubungkan perbaikan kode dengan perbaikan pengujiannya melalui beberapa tahap umpan balik [Ravi dkk., 2026]. Pengujian mempunyai peran penting karena membantu memeriksa apakah perubahan kode tetap memenuhi tugas semula. Hilangnya peringatan pada satu alat tidak cukup untuk menjelaskan seluruh hasil perbaikan. Alat yang digunakan, jenis kelemahan yang diperiksa, dan pengujian fungsi perlu dijelaskan bersama.

2.4.2 Pemisahan agen perbaikan

Model yang membuat kode dapat diminta memperbaiki keluarannya sendiri. Cara ini disebut *self-repair*. Pilihan lainnya adalah menggunakan agen Fixer yang terpisah dari generator. Model lain dapat memberikan cara penyelesaian berbeda, tetapi tetap mungkin memiliki kesalahan yang sama. Oleh karena itu, penggunaan model berbeda belum tentu memberikan hasil yang lebih baik. Manfaatnya perlu dibandingkan dengan penggunaan model yang sama pada tugas dan anggaran yang sebanding. Pasangan model dan jumlah ronde perbaikan penelitian ini belum ditetapkan [VERIFIKASI V09].

Model juga dapat menerima contoh perbaikan dari penyimpanan pengetahuan atau *knowledge store*. Pengambilan contoh yang sesuai disebut *retrieval*. AutoPatch menggunakan informasi CVE untuk membantu pemeriksaan dan perbaikan, sedangkan penelitian multi-tool menyimpan hasil perbaikan yang telah lolos pengujian agar dapat digunakan kembali [Seo dkk., 2025; Sriram dkk., 2026].

Dalam rancangan penelitian ini, Validation bertugas menyimpan pola dan patch yang telah diperiksa. Analyse, Triage, dan Fixer dapat membaca informasi tersebut. Setiap contoh perlu dilengkapi versi kode, masalah yang diperbaiki, dan hasil pengujiannya. Contoh yang berhasil pada satu kasus belum tentu sesuai

untuk kasus lain. Data pengujian juga perlu dipisahkan dari contoh yang diberikan kepada model agar jawaban untuk sampel uji tidak bocor ke proses pemeriksaan [VERIFIKASI V11].

2.4.3 Pemeriksaan ulang dan batas perbaikan

Perbaikan dapat menimbulkan masalah baru pada bagian yang sebelumnya tidak bermasalah. Keadaan ini disebut regresi. Studi pada perbaikan Infrastructure-as-Code menunjukkan bahwa hasil pemeriksaan keamanan dapat memburuk setelah perubahan kode. Penulis juga membedakan regresi yang benar-benar terjadi dari perubahan hasil ukur akibat penataan ulang sumber daya [Agyekum dan Santos, 2026]. Temuan pada Terraform tersebut tidak langsung menunjukkan tingkat risiko pada Python, tetapi menjelaskan alasan perlunya pemeriksaan ulang setelah perubahan.

Pada gate yang direncanakan, patch kembali melalui Sanitizer dan seluruh pemeriksaan yang berlaku. Pengujian yang sebelumnya menemukan masalah dipertahankan untuk memeriksa apakah masalah itu muncul kembali. Pengujian fungsi juga diperlukan agar model tidak menyelesaikan kelemahan dengan menghilangkan tugas program. Untuk sebuah fungsi atau potongan kode, pengujian dapat membutuhkan program pembantu atau *test harness* yang menyiapkan masukan dan memeriksa hasil. Kelayakan cara ini masih perlu diuji [VERIFIKASI V12].

Jumlah perbaikan perlu dibatasi agar proses tidak terus mengulang perubahan yang tidak menyelesaikan masalah. Jika batas tercapai, kode tidak langsung diterima. Arbiter menentukan tindak lanjut sesuai masalah yang masih tersisa. Begitu pula, kode yang belum dapat dijalankan tidak boleh dianggap telah lulus pengujian dinamis hanya karena tidak ada laporan kegagalan.

Sintesis dan Keterbatasan. Hasil alat pemeriksa dan contoh patch dapat membantu model memperbaiki kode. Namun, setiap patch tetap perlu diuji kembali karena dapat mengubah fungsi atau menimbulkan kelemahan baru. Penelitian ini merencanakan Fixer terpisah, pemeriksaan ulang, dan batas

ronde perbaikan. Keberhasilannya dinilai dari keamanan dan fungsi kode setelah perbaikan, bukan hanya dari berkurangnya peringatan alat.

2.5 Ketahanan Gate terhadap Serangan

2.5.1 Serangan pada agen pemeriksa

Agen pemeriksa atau *reviewer* LLM membaca kode dan informasi pendukung untuk menilai keamanan. Informasi tersebut dapat berasal dari pihak yang tidak tepercaya. Penyerang dapat menyisipkan instruksi dalam komentar, docstring, atau teks lain agar model mengabaikan kelemahan. Upaya mengalihkan model melalui instruksi semacam ini disebut *prompt injection*. Pada gate, serangan dapat bertujuan menyembunyikan temuan, menurunkan tingkat keparahan, atau membuat kode rentan memperoleh keputusan pass.

Serangan perlu dibedakan berdasarkan tugas model yang disasar. TAPI menggunakan pemicu dalam komentar untuk memengaruhi kode yang dibuat model [Yang dkk., 2024]. Your AI, My Shell membahas pengalihan editor agentik melalui sumber pengembangan yang telah disisipi instruksi penyerang [Liu dkk., 2026]. Kedua penelitian tersebut menunjukkan risiko dari masukan tidak tepercaya, tetapi tugasnya berbeda dari pemeriksaan keamanan kode. Hasil serangan pada generator atau editor tidak langsung menunjukkan tingkat keberhasilan serangan pada reviewer.

Dalam penelitian ini, batas kemampuan penyerang perlu ditentukan terlebih dahulu. Sebagai contoh, penyerang mungkin hanya dapat mengubah komentar tanpa mengubah perintah program. Skenario tersebut berbeda dari penyerang yang dapat mengubah keluaran alat atau isi knowledge store. Perbedaan akses ini memengaruhi cara serangan diuji dan hasilnya ditafsirkan [VERIFIKASI V13].

2.5.2 Hasil penelitian serangan pada reviewer

Maloyan dan Namiot menunjukkan bahwa penilaian LLM dapat dipengaruhi oleh prompt injection. Mereka membahas penggunaan beberapa model penilai dan penilaian komparatif sebagai arah pertahanan [Maloyan dan Namiot, 2025]. Namun, tugas penilaian umum tidak sama dengan pencarian

kelemahan kode. Beberapa model juga dapat membaca instruksi penyerang yang sama. Oleh karena itu, penambahan reviewer tetap perlu diuji dan belum dapat dianggap cukup untuk mencegah serangan.

Hasil yang berbeda ditemukan dalam penelitian Thornton. Percobaan menggunakan 100 sampel kode rentan dari Python, JavaScript, dan Java yang dinilai oleh delapan model komersial maupun terbuka. Pengaruh komentar serangan tidak signifikan secara statistik pada pengujian berpasangan. Penghapusan komentar justru menurunkan kemampuan deteksi model yang lebih lemah. Sebaliknya, pemberian temuan SAST sebagai petunjuk membantu pemeriksaan [Thornton, 2026]. Hasil tersebut berlaku pada sampel dan jenis komentar yang diuji, sehingga tidak berarti reviewer kebal terhadap seluruh serangan.

Perbedaan hasil ini menunjukkan pentingnya mempertahankan tugas utama ketika menilai pertahanan. Sistem perlu tetap mampu membaca konteks yang dibutuhkan untuk memeriksa kode. Tinjauan ancaman agen juga menekankan bahwa perlindungan, kegunaan, dan waktu pemrosesan perlu dinilai bersama [P. Wang dkk., 2026]. Pertahanan yang membuat model mengabaikan semua informasi tambahan dapat mengurangi serangan, tetapi juga dapat mengurangi kemampuannya menyelesaikan tugas.

2.5.3 Sanitizer dan aturan keputusan

Sanitizer adalah bagian non-LLM yang direncanakan menyiapkan masukan sebelum dibaca reviewer. Bagian ini mengurangi komentar, docstring, atau instruksi tersembunyi yang berpotensi mengalihkan model. Namun, penghapusan informasi belum tentu selalu membantu. Komentar dapat memberikan konteks, sedangkan docstring dan string tertentu dapat digunakan oleh program. Karena itu, masukan yang disederhanakan untuk reviewer perlu dibedakan dari kode asli yang dipindai dan dijalankan. Hubungan nomor baris dengan kode asli juga perlu dipertahankan [VERIFIKASI V14].

Manfaat sanitizer akan dinilai dengan membandingkan gate yang menggunakannya dan gate yang tidak menggunakannya. Pengujian memakai

sampel yang sama, baik tanpa serangan maupun setelah disisipi serangan. Kelemahan kode harus tetap ada pada kedua sampel. Jika perubahan komentar sekaligus menghilangkan kelemahan atau mengubah fungsi program, perbedaan keputusan reviewer tidak lagi menunjukkan pengaruh serangan pada kasus yang sama.

Selain keberhasilan serangan, pengujian perlu mencatat perubahan presisi, recall, dan jumlah kode rentan yang diterima. *Attack success rate* adalah proporsi serangan yang mencapai tujuan pada kasus yang diuji. Perhitungannya perlu menjelaskan kasus mana yang masuk ke penyebut. Kelemahan yang memang tidak terdeteksi sejak awal perlu dibedakan dari kelemahan yang baru terlewat setelah komentar serangan ditambahkan.

Arbiter membatasi keputusan reviewer melalui aturan yang ditetapkan, termasuk larangan membatalkan temuan SAST penting tanpa penanganan yang sesuai. Namun, serangan juga dapat berada di luar komentar. PhantomSkill menunjukkan bahwa perilaku berbahaya dapat disamarkan dalam kode yang tampak memiliki kelemahan biasa [Lin dan Yu, 2026]. Karena itu, aturan YAML, keluaran alat, dan pengetahuan yang digunakan model perlu memiliki batas akses yang jelas. Pemeriksaan dependensi dan paket halusinasi belum termasuk ruang lingkup gate ini [VERIFIKASI V11; VERIFIKASI V13].

Sintesis dan Keterbatasan. Reviewer dapat dipengaruhi oleh masukan yang dibacanya, tetapi keberhasilan serangan bergantung pada model, tugas, dan bentuk serangan. Penelitian ini menguji sanitizer dan aturan arbiter untuk melihat pengaruhnya terhadap keputusan gate. Hasil yang tidak menunjukkan peningkatan tetap perlu dilaporkan. Pengujian juga harus memastikan bahwa pertahanan tidak mengurangi kemampuan menemukan kelemahan pada kode tanpa serangan.

2.6 Penelitian Terdahulu dan Posisi Penelitian

2.6.1 Perbandingan penelitian terdahulu

Penelitian terdahulu telah membahas pembuatan kode, pemeriksaan kerentanan, kerja sama agen, dan perbaikan otomatis. Tabel berikut merangkum

pendekatan, data pengujian, hasil, serta keterbatasan yang berkaitan dengan penelitian ini. Angka hasil hanya dicantumkan jika konteks utamanya telah diperiksa. Bagian yang belum cukup jelas tetap diberi tanda verifikasi. Keterangan tentang keterbatasan dibatasi pada bagian paper yang telah dibaca dan belum berarti suatu kemampuan tidak tersedia dalam seluruh sistemnya.

Tabel 1: Perbandingan penelitian terdahulu untuk multi-agent security gate

Studi	**Pendekatan**	**Dataset / objek**	**Hasil utama**	**Keterbatasan**
AutoSafeCoder; Nunez dkk. (2024)	Coding, reviewer LLM, fuzzing; iteratif	SecurityEval; HumanEval; GPT-4o	Kode rentan 59/121 menjadi 44/121; pass@1 sedikit turun	Reviewer statis berbasis LLM; Bandit sebagai penilai eksperimen. Ketahanan reviewer belum menjadi evaluasi utama.
Teaming-Up LLMs; Sabir dkk. (2026)	Beberapa model, CodeQL, pemeriksaan dan perbaikan kode	SecLLMEval dan SecLLMHolmes; C/Python	Hibrida dilaporkan lebih konsisten dari kolaborasi LLM saja	Sudah mencakup pembuatan sampai perbaikan kode. Perbedaan angka hasil perlu diperiksa [VERIFIKASI V05].
ReasonVul; Peng dkk. (2026)	Tiga gaya penalaran dan debat	PrimeVul; JITVUL; model berbeda	389/542 konflik diselesaikan benar pada analisis PrimeVul	Berfokus pada deteksi kode rentan dan perbaikannya; belum menilai keputusan penerimaan kode oleh gate.
VulTrial; Widyasari dkk. (2025)	Pembagian peran seperti ruang sidang	Benchmark deteksi; antara lain GPT-4o	Perbaikan deteksi pada konfigurasi penulis	Tugas hakim dan juri perlu dibandingkan dengan aturan pass/ revise/block pada gate.
MAVUL; Y. Li dkk. (2025)	Pemeriksaan konteks kode dan diskusi antaragen	Dataset kerentanan berpasangan	Peningkatan dilaporkan; angka rinci tidak dikutip	Abstrak sumber asli sudah diperiksa; pengaturan dan hasil percobaan masih perlu dibaca [VERIFIKASI V10].
QASecClaw; Ameen dkk. (2026)	Semgrep, penyaringan LLM, penggabungan bukti	OWASP v1.2; 2.740 kasus Java; Qwen 3.5 Plus	False positive 560 menjadi 64	Recall ikut turun. Pengujian dinamis tidak dijalankan dalam percobaan; fokus pada penyaringan temuan.
SAST-Genius; Agrawal dan Ahi (2025)	Semgrep dan analisis LLM	Pengujian dalam laporan penulis	False positive dilaporkan berkurang	Dataset dan pengaturan percobaan masih perlu diperiksa [VERIFIKASI V06].
ZeroFalse; Iranmanesh dkk. (2025)	Jejak aliran data dan prompt khusus CWE	OWASP Java Benchmark; OpenVuln	Keseimbangan presisi–recall dilaporkan membaik	Memeriksa peringatan pada kode yang sudah ada; perlu diuji kembali pada kode Python hasil LLM.
IRIS; Z. Li dkk. (2025)	LLM membantu pelacakan data dalam CodeQL	120 kerentanan CWE-Bench-Java; GPT-4	55 terdeteksi dibanding 27 pada CodeQL	Menggunakan proyek Java; hasilnya belum tentu sama pada potongan kode Python hasil LLM.
FDSP; Alrashedy dkk. (2024)	Umpan balik Bandit untuk patching	PythonSecurity Eval; kode Python hasil LLM	Umpan balik eksternal mendukung perbaikan	Belum membuktikan manfaat model perbaikan yang berbeda; hasil perbaikan perlu diuji kembali.

Studi	Pendekatan	Dataset / objek	Hasil utama	Keterbatasan
RAG + Multi-Tool; Sriram dkk. (2026)	Retrieval, GCC, CodeQL, KLEE	3.242 program C/C++; dua model	Pada CodeLlama-7B, cacat keamanan 58,55% menjadi 22,19%	Model memperbaiki kode C/C++ pendek; cara pengujiannya perlu disesuaikan untuk Python.
Maloyan dan Namiot (2025)	Prompt injection pada penilai LLM	Beberapa tugas penilaian	Penilaian model dapat dipengaruhi oleh serangan	Tugas penilaian berbeda dari pemeriksaan keamanan kode; hasilnya tidak dapat langsung disamakan.
Thornton (2026)	Serangan komentar dan evaluasi pertahanan	100 sampel rentan; tiga bahasa; delapan model	Pengaruh komentar tidak signifikan pada percobaan berpasangan	Penghapusan komentar dapat menurunkan deteksi. Sampel rentan saja belum cukup untuk menilai false positive.
CodeSecEval / SecAwareCoder; J. Wang dkk. (2026)	Pengenalan ancaman dan hasil pengujian kode	v3: 255 tugas Python; 77 CWE	Versi baru menambah dataset dan cara pembuatan kode	Versi terbaru dari sumber yang sama; rincian sistem dan pengujian serangan masih perlu dibaca [VERIFIKASI V04].

2.6.2 Posisi penelitian

AutoSafeCoder dan Multi-LLMSecCodeEval menunjukkan bahwa pembuatan, pemeriksaan, dan perbaikan kode dapat dihubungkan dalam satu alur. AutoPatch juga menggunakan beberapa agen dan informasi kerentanan untuk membantu perbaikan. Sementara itu, QASecClaw mendekati kebutuhan Triage dan Validation karena menggabungkan SAST dengan pemeriksaan konteks oleh LLM. Dengan adanya penelitian tersebut, bentuk umum alur generate–review–repair tidak digunakan sebagai kebaruan tersendiri.

Cara mengambil keputusan juga telah dibahas dalam beberapa bentuk. ReasonVul menggunakan debat ketika pendapat agen berbeda, VulTrial membagi peran seperti ruang sidang, dan Refute-or-Promote menggunakan pemeriksaan bertahap sebelum temuan diterima. Revisi CodeSecEval melalui SecAwareCoder turut menambah pembanding untuk pembuatan kode berdasarkan ancaman dan hasil pengujian. Oleh karena itu, perbedaan penelitian ini perlu ditentukan dari cara komponen digunakan dan diuji, bukan hanya dari keberadaan agen atau arbiter.

Dari pembahasan sebelumnya, terdapat tiga kebutuhan yang saling berkaitan. Pertama, hasil pemeriksaan perlu memiliki dasar yang jelas agar temuan yang benar tidak dibuang saat triase. Kedua, kode yang diperbaiki perlu diperiksa ulang agar fungsi tetap berjalan dan kelemahan baru tidak muncul.

Ketiga, perlindungan terhadap serangan reviewer perlu diuji bersama kemampuan deteksi. Ketiga kebutuhan ini menjadi dasar pengembangan gate pada penelitian ini.

Gate yang direncanakan menghubungkan generator dengan Sanitizer, Analyse, Triage, Testing, Validation, dan arbiter. Ketika kode perlu diperbaiki, Fixer menghasilkan patch yang kembali melalui pemeriksaan. Validation direncanakan menyimpan contoh pola serta patch yang telah diperiksa dalam knowledge store. Namun, tugas rinci Testing LLM, kelayakan pengujian dinamis, pasangan model, dan batas ronde perbaikan belum diputuskan. Uraian tersebut masih berupa rencana sistem, bukan hasil penerapan yang telah terbukti [VERIFIKASI V09; VERIFIKASI V11; VERIFIKASI V12].

Penelitian ini diarahkan untuk menggabungkan pemeriksaan, aturan pass/revise/block, dan perbaikan kode, lalu menguji hasilnya pada Python. Manfaat sanitizer dan penggunaan model Fixer berbeda juga akan dinilai melalui percobaan pembanding. Pengujian direncanakan memakai LLM tunggal, SAST saja, serta SAST dengan LLM sebagai baseline. Pengaruh tiap komponen dapat dilihat dengan menjalankan sistem tanpa komponen tersebut dan membandingkan hasilnya pada sampel yang sama.

Ukuran yang diperhatikan mencakup precision, recall, dan F1 per CWE, jumlah kode rentan yang diterima, keberhasilan patch, serta perubahan fungsi setelah perbaikan. Untuk serangan reviewer, pengukuran memakai pasangan kode tanpa serangan dan kode yang diserang. Penggunaan token dan waktu pemrosesan dihitung sepanjang alur, termasuk pengulangan dan kegagalan. Label acuan perlu diperiksa secara terpisah agar penilaian akhir tidak hanya mengulang keputusan alat yang digunakan di dalam gate [VERIFIKASI V15].

Penelusuran literatur ini belum mencakup seluruh penelitian yang mungkin berkaitan. Beberapa sumber baru diperiksa abstraknya, sehingga kode dan pengaturan percobaan pembanding terdekat masih perlu dibaca. Pemeriksaan tersebut diperlukan untuk memastikan bagian mana yang sudah ditangani dan bagian mana yang masih dapat dikembangkan [VERIFIKASI V04;

VERIFIKASI V10]. Klaim penelitian dibatasi pada penggabungan komponen dan pengujiannya dalam ruang lingkup yang ditetapkan.

Sintesis dan Keterbatasan. Penelitian ini mengembangkan gate untuk memeriksa dan memperbaiki kode hasil LLM dengan aturan keputusan yang jelas. Evaluasinya mencakup kemampuan deteksi, hasil perbaikan, ketahanan terhadap serangan reviewer, serta biaya penggunaan model. Pembahasan dibatasi pada Python dan lima sampai delapan CWE yang akan dipilih. Dengan batasan tersebut, hasil penelitian diharapkan menjelaskan manfaat dan kekurangan gate pada kondisi pengujian, tanpa menyatakan bahwa kode yang diterima telah bebas dari seluruh risiko keamanan.